

When Feasible Solutions Become a Trap: Initialization Strategies for EDAs in the Traveling Tournament Problem

Guilherme Nakahata^{1,2}[0000–0001–6469–8653], Claus
Aranha¹[0000–0003–1390–7536], and Florian Richoux²[0000–0003–4693–379X]

¹ University of Tsukuba, Japan

² National Institute of Advanced Industrial Science and Technology, Japan

Abstract. The Traveling Tournament Problem is a highly constrained scheduling problem in which obtaining feasible and good quality schedules remains difficult for optimization methods. In this work, we study how initialization and feasibility influence the behavior of Estimation of Distribution Algorithms (EDA) using a Univariate EDA and a Tree-based EDA. Experiments were conducted using feasible, partially feasible, and infeasible initial solutions under different penalty intensities, and the behavior of local search was also investigated. Initialization had a stronger influence on the search process than the penalty intensity. Most of the improvements observed in the hybrid approaches originate from the local search component, particularly for the larger benchmark instances. These results provide evidence that learning effective probabilistic models for highly constrained scheduling problems remains challenging, and help clarify the importance of initialization and local search in this process.

Keywords: Traveling Tournament Problem (TTP) · Estimation of Distribution Algorithm (EDA) · Feasible Initialization · Constraint Optimization.

1 Introduction

Sports scheduling is a challenging task due to its combinatorial nature and the need to satisfy multiple constraints. The Traveling Tournament Problem (TTP) is a well-known benchmark problem that captures aspects of both the Traveling Salesman Problem (TSP) and sports scheduling [7, 24]. The TTP involves scheduling n teams over $2(n - 1)$ slots, with each team playing one match per round against a different opponent. The objective is to minimize the total travel distance while adhering to a set of restrictive constraints:

- (i) Double round-robin (*doubleRoundRobin*): where each team plays every other team twice (home and away);
- (ii) Maximum streak (*maxStreak=k*): limiting the number of consecutive home or away games (usually set to $k=3$); and

- (iii) No repeats (*noRepeat*): preventing teams from facing the same opponent in consecutive slots.

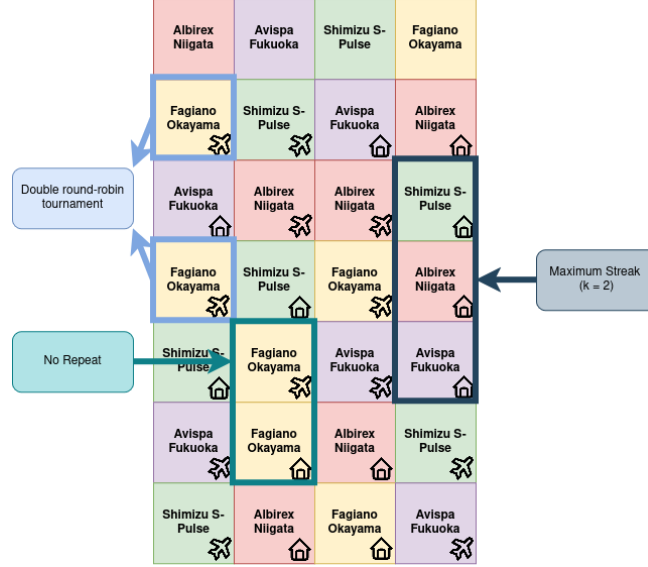


Fig. 1: An example of infeasible TTP instance. The rows indicate slots, while the columns indicate the matches of each team. The airplane/house icon indicates an "away" or "home" game, respectively. Here, we consider $k = 2$ for the maxStreak constraint.

These constraints are illustrated in Figure 1. Compared to the TSP, whose only requirement is to visit each city exactly once, the TTP introduces additional structural and temporal dependencies, resulting in a highly constrained and complex search space.

Consequently, generating valid solutions is already difficult, and improving them during the optimization process becomes even more challenging [16, 12, 17]. These constraints limit the applicability of standard variation operators and complicate the behavior of population-based methods such as Estimation of Distribution Algorithms (EDAs) [21, 17].

Besides, local search can refine candidate solutions and enable more effective navigation of the feasible search space. How local search interacts with EDAs in TTP under these constraints remains unclear.

In this work, we investigate these questions using a Univariate EDA and a Tree-based EDA for the Traveling Tournament Problem. We analyzed the impact of feasible, partially feasible, and infeasible initial populations under different penalty intensities, evaluated the contribution of local search to this optimization process and compared the resulting hybrid approaches against standalone local

search. The results indicate that initialization has a stronger influence on the search process than penalty intensity, while local search is responsible for most of the improvements in solution quality.

2 Related Work

The TTP is a sports timetabling problem introduced by Easton et al. [9] and has been extensively studied in the literature, with a variety of approaches proposed to address its combinatorial complexity [3, 4]. In Verduin et al. [21] showed that generating feasible solutions for the TTP is difficult, and that evolutionary computation methods often struggle with its constraints.

More recently, attention has been given to the role of initialization in TTP. Nakahata et al. [17] proposed a linear time method capable of generating a large number of feasible schedules, up to $(2n - 1) \cdot 2n \cdot n!$ for n teams. This result is relevant for its efficiency, but also because it provides access to a large set of feasible starting solutions, which can influence how population-based algorithms work during the search.

In the work of Goerigk et al. [11], the authors presented a heuristic approach based on Iterative Deepening A* (IDA*), incorporating techniques to reduce the search space. Another approach for the TTP [14] formulates it as a constrained optimization problem, explicitly distinguishing between feasibility and optimality. This method integrates stochastic local search (SLS) with local search techniques such as Variable Neighborhood Search (VNS) and Simulated Annealing (SA) to operate within feasible regions of the search space.

The TTP has also been addressed as a bi-objective optimization problem [22], where both travel distance and constraint violations are minimized. In this setting, algorithms such as NSGA-II are combined with local search techniques, the results highlight the effectiveness of local search in finding feasible high-quality solutions.

Meanwhile, Estimation of Distribution Algorithms (EDAs) are stochastic optimization methods that explore the search space by sampling from an explicit probabilistic model learned from high-quality solutions [13, 2]. EDAs have been successfully applied to several combinatorial and continuous optimization problems.

EDAs have been applied to a variety of domains, such as the job scheduling problem [8], many-objective optimization [20], the supermarket location problem [25], and stacked autoencoder optimization [23].

Despite these advances, EDAs remain underexplored for the TTP. More precisely, the impact of initialization strategies on the learned probabilistic model and its interaction with local search has received little attention. This is particularly relevant in highly constrained problems, where feasibility and exploration affect not only search dynamics but also how local search acts during optimization.

3 Traveling Tournament Problem

In this work, we adopt a match-based representation for the Traveling Tournament Problem (TTP). Each match is uniquely identified by an integer index $m \in \{0, \dots, M-1\}$, where $M = n(n-1)$ is the total number of matches. Each index m is mapped to an ordered pair of teams (i, j) , representing a directed match in which team i plays at home against team j .

A solution to the Traveling Tournament Problem is represented as a vector $x = (x_0, \dots, x_{M-1})$, where each variable x^m denotes the slot assigned to match m . Each variable takes values in the domain $\{0, \dots, S-1\}$, where $S = 2(n-1)$ corresponds to the total number of slots. The objective function applied in this work penalizes constraint violations during the optimization process and is defined as:

$$f(x) = \text{cost}(x) + \lambda \cdot \text{violations}(x) \quad (1)$$

where $\text{cost}(x)$ represents the total travel distance of all teams, calculated from the sequence of away games, and $\text{violations}(x)$ are the number of constraint violations in the schedule.

The penalization coefficient λ is defined as the sum of all distances in the distance matrix:

$$\lambda = \rho \cdot \sum_{i=1}^n \sum_{j=1}^n d_{ij} \quad (2)$$

where d_{ij} represents the distance between teams i and j . The coefficient λ is computed from the distance matrix so that the penalty remains proportional to the travel costs of the instance. The parameter ρ controls how strongly constraint violations affect the fitness value. Larger values of ρ give more emphasis on feasibility, while smaller values allow the search to explore infeasible solutions.

4 Algorithm

The use of an Estimation of Distribution Algorithm in this work is motivated by the strong dependencies between matches in the Traveling Tournament Problem. Even small changes in a schedule may easily introduce multiple constraint violations, as illustrated in Figure 1.

The initialization method proposed by Nakahata et al. [17] enables the generation of feasible solutions and as the EDA learns probabilistic patterns from promising solutions, it may exploit this type of structured initialization. However, the use of EDAs for the TTP is still not widely explored, especially in combination with feasible initialization strategies.

4.1 Univariate Estimation of Distribution Algorithm

Estimation of Distribution Algorithm (EDA) is a class of evolutionary algorithms that replaces traditional variation operators with probabilistic modeling. While the Genetic Algorithm (GA) applies crossover and mutation, EDA iteratively builds a probabilistic model from good solutions and generates new candidates by sampling from this model [15].

In this work, we employ a Univariate Estimation of Distribution Algorithm (EDA), in which each variable is modeled independently through its marginal distribution. Algorithm 1 presents the proposed EDA combined with a local search procedure. In each generation, the population P is sorted according to the fitness function, and an elite set $\mathcal{E}$ is selected using the SELECTELITE method and the elite rate α .

The elite set $\mathcal{E}$ is used to estimate the probabilistic model, and a subset of these elite solutions is directly carried to the next generation according to the retained rate β . The remaining individuals are generated by sampling from the learned probabilistic model. This strategy is inspired by approaches in the EDA literature, such as extreme elitism [10].

Algorithm 1 Univariate Estimation of Distribution Algorithm with Local Search

Require: population size N , maximum generations G , elite rate α , retained rate β , local search probability p_{ls} , interval τ

```

1: Initialize population  $P$ 
2: for  $g = 0$  to  $G - 1$  do
3:    $\mathcal{E} \leftarrow \text{SELECTELITE}(P, \alpha)$ 
4:    $\theta \leftarrow \text{ESTIMATEMODEL}(\mathcal{E})$ 
5:    $P \leftarrow \text{RETAINELITE}(\mathcal{E}, \beta)$ 
6:   while  $|P| < N$  do
7:      $x \sim P(\theta)$ 
8:     if  $g \bmod \tau = 0$  then
9:       if  $\text{RAND}() < p_{ls}$  then
10:         $x \leftarrow \text{LOCALSEARCH}(x)$ 
11:       end if
12:     end if
13:     Evaluate  $f(x)$ 
14:      $P \leftarrow P \cup \{x\}$ 
15:   end while
16: end for
17: return  $P$ 

```

The elite solutions are used to estimate a probabilistic model through the ESTIMATEMODEL procedure. Each variable x^m represents a match and can take a slot $s \in \{0, \dots, S - 1\}$. The assignment probabilities are computed based on the frequencies observed in the elite set $\mathcal{E}$.

The frequency of assigning slot s to match m is defined as:

$$C_{m,s} = \sum_{i \in \mathcal{E}} \mathbf{1}(x_i^m = s) \quad (3)$$

where $C_{m,s}$ is the number of elite solutions in which match m is assigned to slot s . These frequencies are converted into probabilities using Laplace smoothing with $\eta = 1$:

$$P_{m,s} = \frac{C_{m,s} + \eta}{|\mathcal{E}| + \eta \cdot S} \quad (4)$$

where $P_{m,s}$ represents the probability of assigning slot s to match m . New candidate solutions x are generated by independently sampling from the probabilistic model, each variable is sampled independently from a categorical distribution defined by $P_{m,s}$:

$$x^{(m)} \leftarrow \text{Categorical}(P_{m,0}, \dots, P_{m,S-1}) \quad (5)$$

This sampling process assumes independence between variables, allowing the joint distribution to be factorized as:

$$P(x) = \prod_m P(x^{(m)}) \quad (6)$$

A local search procedure is applied every τ generations with probability p_{ls} to improve sampled solutions. This process is repeated until the population reaches size N , and the algorithm proceeds for G generations.

4.2 Tree-based Estimation of Distribution Algorithms

Tree-based Estimation of Distribution Algorithms (EDA-Tree) extends EDA by modeling dependencies between variables with a tree structure, capturing interactions between decision variables [1]. In this work, the model is constructed using a maximum spanning tree with Kruskal's algorithm based on pairwise mutual information.

The Tree-based Estimation of Distribution Algorithms (EDA-Tree) algorithm extends the EDA by explicitly modeling pairwise dependencies among variables. The probabilistic model is represented as a dependency tree learned from mutual information statistics using the Chow-Liu algorithm [5]. The resulting tree defines a factorization of the joint distribution in which each variable depends on at most one parent variable.

The EDA-Tree is based on the EDA (Algorithm 1), including SELECTELITE and LOCALSEARCH, but differs in how the probabilistic model is estimated based on conditional dependencies defined by the tree, as shown in Algorithm 2.

Once the elite set $\mathcal{E}$ is selected, the probabilistic model is estimated by computing marginal and joint probabilities. The marginal probability $P_m(s)$ represents how often a variable x_m takes value s in the elite solutions and is defined as:

Algorithm 2 Tree Estimation of Distribution Algorithm with Local Search

Require: population size N , maximum generations G , elite rate α , retained rate β , local search probability p_{ls} , interval τ

```

1: Initialize population  $P$ 
2: for  $g = 0$  to  $G - 1$  do
3:    $\mathcal{E} \leftarrow \text{SELECTELITE}(P, \alpha)$ 
4:    $M \leftarrow \text{COMPUTEMARGINALS}(\mathcal{E})$ 
5:    $MI \leftarrow \text{COMPUTEMUTUALINFORMATION}(\mathcal{E}, M)$ 
6:    $T \leftarrow \text{BUILDMAXSPANNINGTREE}(\text{KRUSKAL})(MI)$ 
7:    $\theta \leftarrow \text{ESTIMATECONDITIONALPROBABILITIES}(\mathcal{E}, T)$ 
8:    $P \leftarrow \text{RETAINELITE}(\mathcal{E}, \beta)$ 
9:   while  $|P| < N$  do
10:     $x \sim P(\theta, T)$ 
11:    if  $g \bmod \tau = 0$  then
12:      if  $\text{RAND}() < p_{ls}$  then
13:         $x \leftarrow \text{LOCALSEARCH}(x)$ 
14:      end if
15:    end if
16:    Evaluate  $f(x)$ 
17:     $P \leftarrow P \cup \{x\}$ 
18:  end while
19: end for
20: return  $P$ 

```

$$P_m(s) = \frac{1}{|\mathcal{E}|} \sum_{k \in \mathcal{E}} \mathbf{1}(x_k^{(m)} = s) \quad (7)$$

Similarly, the joint probability $P_{m,n}(s_m, s_n)$ captures how often two variables take specific slots simultaneously:

$$P_{m,n}(s_m, s_n) = \frac{1}{|\mathcal{E}|} \sum_{k \in \mathcal{E}} \mathbf{1}(x_k^{(m)} = s_m, x_k^{(n)} = s_n) \quad (8)$$

The dependency structure is represented as a maximum spanning tree, constructed using Kruskal's algorithm based on pairwise mutual information. The mutual information between variables x_m and x_n is defined as:

$$I(m, n) = \sum_{s_m, s_n \in \mathcal{S}} P_{m,n}(s_m, s_n) \log \frac{P_{m,n}(s_m, s_n)}{P_m(s_m)P_n(s_n)} \quad (9)$$

Using these values, a maximum spanning tree is constructed, where each node corresponds to a decision variable x_m . One node is selected as the root x_r , and each non root node depends on its parent $\pi(m, T^*)$ in the tree.

The resulting probabilistic model factorizes according to the tree structure:

$$P(x) = P(x_r) \prod_{m \neq r} P(x_m \mid x_{\pi(m, T^*)}) \quad (10)$$

where $P(x_r)$ is the marginal probability of the root node and $P(x_m \mid x_{\pi(m, T^*)})$ represents the conditional probability of each variable given its parent in the tree. These probabilities are estimated from the elite set using Laplace smoothing with $\eta = 1$ and correspond to the parameters θ , which are used to sample new candidate solutions in Algorithm 2.

4.3 Local Search

To improve solution quality, a local search procedure is periodically applied during the evolutionary process. The local search is executed every τ generations and each newly generated solution is refined with probability p_{ls} .

The local search adopted in this work is provided by the GHOST framework [19]. The neighborhood is defined by all solutions obtained by changing the value of a single variable, in the TTP representation used here, each variable corresponds to a match and its value represents the assigned slot.

The search focuses on the variable with the largest projected error, associated with the largest number of current constraint violations. The selected variable is then reassigned to the slot that generates the largest reduction in the global error.

This implementation supports two tabu mechanisms, the first temporarily marks the selected variable as tabu after each move and the second marks variables that lead to local optima. These variables remain tabu for a number of iterations equal to the number of variables in the problem for small instances, or this number divided by 5 for larger instances. Algorithm 3 presents the Adaptive Search procedure implemented in GHOST.

Algorithm 3 Adaptive Search implemented in GHOST

Require: initial solution x , maximum iterations $I_{\max}$

- 1: Initialize Solution x
 - 2: Initialize tabu list
 - 3: **for** $i = 0$ to $I_{\max} - 1$ **do**
 - 4: Compute constraint errors of x
 - 5: For each variable v , sum the errors of constraints where v is involved
 - 6: $v \leftarrow$ variable with the highest projected error that is not tabu
 - 7: Compute the errors for alternative values of v
 - 8: $s^* \leftarrow$ value that produces the largest error reduction
 - 9: **if** s^* improves the current configuration **then**
 - 10: Assign $v \leftarrow s^*$
 - 11: **else**
 - 12: Mark v as tabu
 - 13: Move randomly
 - 14: **end if**
 - 15: **end for**
 - 16: **return** x
-

Further details on neighborhood evaluation can be found in the original GHOST framework, which was used without modification in this study.

4.4 Initial Solution

The quality of the initial population can significantly affect the convergence behavior of EDA. We consider three initialization strategies to investigate how starting from feasible, partially feasible, or random solutions affects the performance of the probabilistic models and the local search. The strategies are:

- (i) **Alternating Circle Method [17]**, an improved variation of the Circle method that generates fully feasible solutions satisfying all three TTP constraints in linear time;
- (ii) **Circle method [6, 18]**, which generates initial schedules via circular team rotations. It ensures the round-robin structure and avoids immediate repeats, but does not guarantee the maximum streak constraint, resulting in only partially valid TTP solutions;
- (iii) **Random**, a simple exploratory strategy where each match is assigned a random slot, without guaranteeing any feasibility.

5 Parameter Sensitivity

We perform a sensitivity analysis to evaluate the impact of algorithm parameters: population size (N), elite rate (α), and retained rate (β). The evaluated values were $N \in \{250, 500, 1000\}$, $\alpha \in \{5\%, 10\%, 20\%\}$, and $\beta \in \{25\%, 50\%, 75\%\}$. The analysis was performed on the medium sized instance with 12 teams (NL12) [9], using random initialization and the Univariate EDA (1) for 500 generations.

For each parameter configuration, the algorithm was executed for 30 independent runs, the performance was evaluated using the final best fitness value, and the convergence curves were aggregated according to the population size (250, 500, and 1000). As shown in Figure 2, larger population sizes produced better final solutions under the same fitness evaluation budget.

We also evaluated three values for both the elite rate (α) and retained rate (β), using three different selection intensities. Figure 3 summarizes the results, where each cell shows the average fitness over 30 independent runs. Overall, larger values of α produced better results and in contrast, no clear pattern was observed for β , which controls the proportion of elite individuals carried over to the next generation.

These results alone do not explain the algorithm’s behavior, motivating a deeper analysis in Section 5.1.

5.1 Role of Initialization and High Rates

To further analyze the interaction between parameter configurations and initialization strategies, we conducted an additional experiment focusing on elite

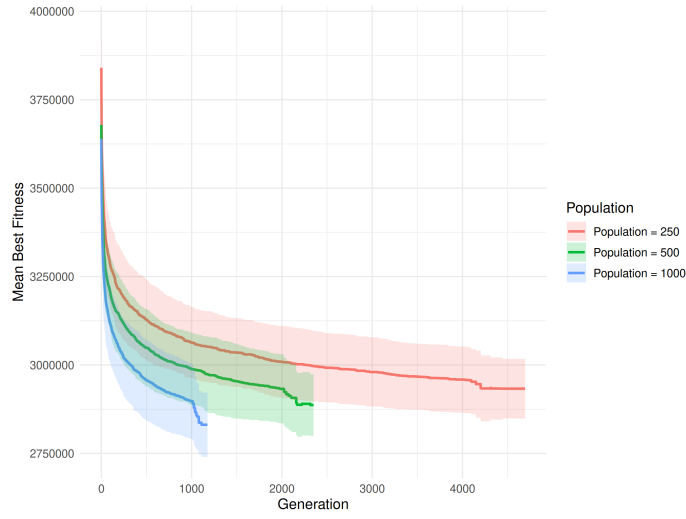


Fig. 2: Convergence behavior of EDA on NL12 with varying population sizes.

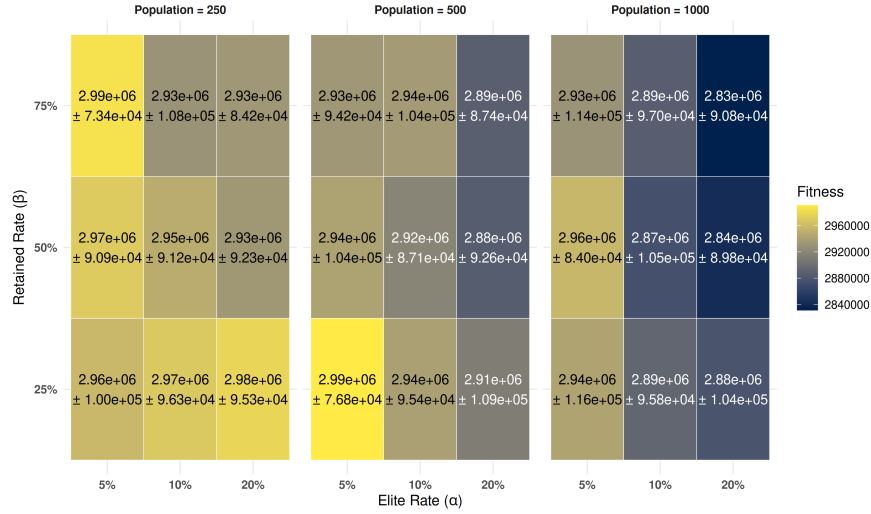


Fig. 3: Average final fitness for different parameter configurations.

and retained rates for NL12, while fixing the population size at $N = 1000$ as this value produced the best results in Section 5. We evaluated four configurations, summarized in Table 1. The four configurations were selected from the best performed configuration identified in the sensitivity analysis to extreme settings. These configurations were selected to analyze the interaction between initialization and selection pressure.

Table 1: Evaluated parameter configurations.

Configuration	Elite Rate (α)	Retained Rate (β)
Config 1	20%	25%
Config 2	50%	75%
Config 3	70%	90%
Config 4	90%	100%

Figure 4 shows the results for two initialization strategies: Random and Circle, the Alternating initialization produced exactly the same fitness value (187,308) for all parameter configurations across all runs. Extreme values in α and β sometimes result in worse performance when compared to moderate configurations, as shown in Figure 3. The initialization strategy has the strongest impact on performance between Alternating and Random/Circle initialization, often masking the effect of the parameter configurations.

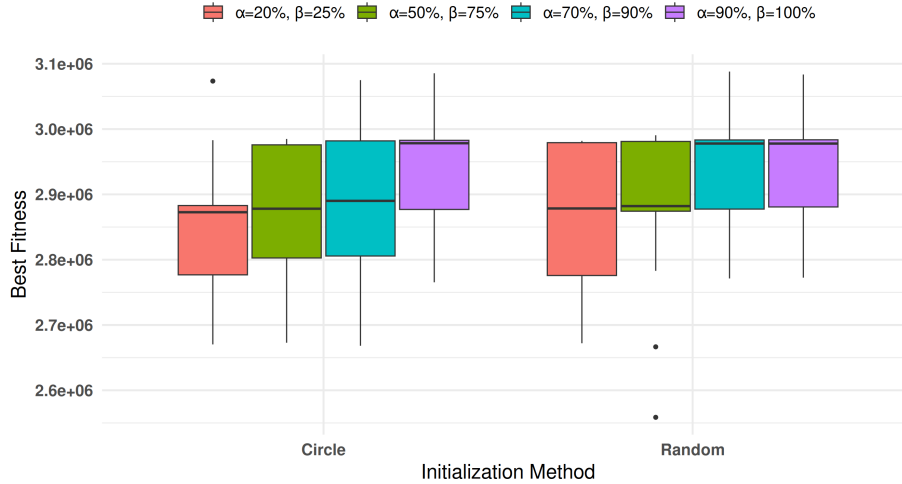


Fig. 4: Final fitness distribution across initialization strategies and parameter configurations. Lower fitness values indicate better solutions.

Random and Circle initializations start from infeasible or partially feasible solutions, requiring the algorithm to explore the search space and reduce constraint violations. In these cases, the probabilistic model and parameter configurations have a more relevant function, but the performance remains variable due to the difficulty of reaching feasible solutions.

These results show that parameter sensitivity alone cannot explain the behavior of EDA in the TTP, motivating the experiments described in Section 6.

6 Results

The experiments were conducted on the NL benchmark instances introduced by Trick [9]. The algorithm parameters were selected based on the sensitivity analysis presented in Section 5 and 5.1. The population size was fixed at $N=1000$, since it produced the best results in the sensitivity analysis (Figure 3) and based on the initialization experiments (Figure 4), we used $\alpha = 20\%$ and $\beta = 25\%$ in the remaining experiments. All configurations were evaluated using a fixed budget of 10 million fitness evaluations and repeated over 30 independent runs. The code and generated data are available online ³.

Our initial analysis focuses on the impact of feasible, partially feasible, and infeasible initialization strategies on the search behavior of EDA and EDA-Tree. While the initialization method influenced the feasibility and exploration patterns observed during the search, it did not explain the differences in performance across configurations.

To further investigate these differences, we extended the analysis by varying the penalty intensity parameter ρ (Equation 2). This allows us to evaluate how different levels of constraint pressure interact with the initialization strategies and affect solution quality, constraint violations, and population diversity. We also examine the role of local search and the ability to improve the solutions generated by the probabilistic models.

6.1 Initialization, Exploration, and Feasibility

In this experiment, we compare the Random, Circle, and Alternating initialization strategies in both EDA and EDA-Tree. The objective is to understand how the feasibility of the initial population influences the search process and the final solution quality.

Figure 5 presents the evolution of the mean number of constraint violations during the search. Both EDA and EDA-Tree struggled to reduce infeasibility, under all initialization strategies. For Random and Circle methods, the number of constraint violations remained high during the entire optimization process. Only a small reduction can be observed for EDA during the early stages of the search, showing that the probabilistic models have a limited ability to improve feasibility and guide the population to feasible regions.

A different behavior was observed for the Alternating initialization. Since this method generates feasible schedules, all runs started from feasible solutions. However, the average number of constraint violations remained nearly unchanged during the search, and both EDA and EDA-Tree had difficulty escaping the region defined by the initial solution, this behavior is evident in all instances.

Table 2 shows the final fitness values obtained using Random and Circle initialization. EDA consistently achieved better fitness values than EDA-Tree, the differences between Random and Circle were limited. In some instances Random produced the best results, while in others Circle obtained lower fitness values.

³ <https://github.com/GuilhermeNkht/EDA-TREE-EDA-TTP>

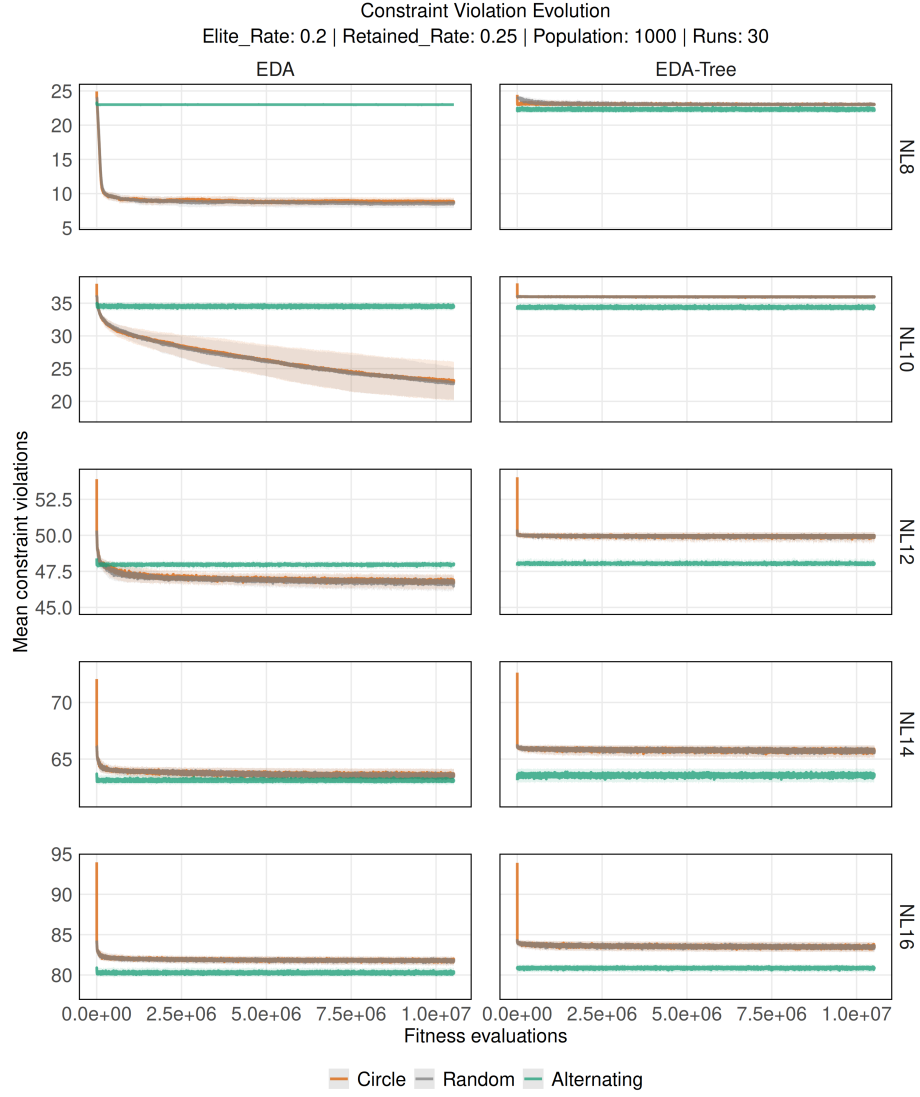


Fig. 5: Evolution of mean constraint violations for all NL instances with $\rho = 1.0$.

More importantly, neither initialization strategy was able to produce feasible solutions during the optimization process.

As the instance size increases, the difference between EDA and EDA-Tree becomes smaller. For NL14 and NL16, both algorithms converge to solutions of similar fitness, indicating that the increased difficulty of the search limits the advantages provided by the differences between the two probabilistic models.

Table 2: Final fitness results for Random and Circle initializations.

Instance	Algorithm	Initialization	Best Fitness	Std. Dev. of	Mean Best Fitness
			Across Runs	Best Fitness	Across Runs $\pm$ 95% CI
NL8	EDA	Circle	144,564	17,947.02	169,959.13 $\pm$ 6,701.53
		Random	111,256	21,089.68	161,747.60 $\pm$ 7,875.02
	EDA-TREE	Circle	369,897	13,411.78	401,698.73 $\pm$ 5,008.04
		Random	369,559	14,682.60	401,790.57 $\pm$ 5,482.57
NL10	EDA	Circle	640,288	77,487.88	734,743.77 $\pm$ 28,934.45
		Random	642,895	71,437.71	720,987.13 $\pm$ 26,675.28
	EDA-TREE	Circle	982,292	37,438.82	1,053,687.63 $\pm$ 13,979.89
		Random	983,047	36,418.30	1,062,172.47 $\pm$ 13,598.82
NL12	EDA	Circle	2,562,170	70,020.56	2,669,017.67 $\pm$ 26,146.11
		Random	2,561,070	68,441.87	2,664,756.67 $\pm$ 25,556.62
	EDA-TREE	Circle	2,769,860	55,532.61	2,839,744.00 $\pm$ 20,736.22
		Random	2,567,580	97,236.90	2,798,109.00 $\pm$ 36,308.85
NL14	EDA	Circle	6,305,860	250,365.36	7,060,942.00 $\pm$ 93,487.96
		Random	6,288,900	230,492.66	7,073,106.67 $\pm$ 86,067.38
	EDA-TREE	Circle	7,084,240	124,666.41	7,300,484.00 $\pm$ 46,551.20
		Random	6,680,900	191,104.75	7,294,111.33 $\pm$ 71,359.69
NL16	EDA	Circle	12,503,300	357,524.93	13,408,680.00 $\pm$ 133,502.00
		Random	12,219,500	344,529.99	13,456,756.67 $\pm$ 128,649.61
	EDA-TREE	Circle	12,784,900	354,161.75	13,818,130.00 $\pm$ 132,246.17
		Random	12,487,500	395,387.11	13,676,143.33 $\pm$ 147,639.97

The results obtained with Alternating initialization are shown in Table 3. In all runs, the final solution was identical to the initial feasible schedule generated by the Alternating method, consequently, both algorithms remained trapped in the initial feasible solution.

The initialization strategy strongly affected both feasibility and search progression, determining the behavior of the probabilistic models during the optimization process. However, it remains unclear whether these effects are mainly due to the characteristics of the initialization methods or are amplified by the feasibility pressure imposed by the penalty function (Equation 2). The next section analyzes the effect of different penalty intensities on both algorithms.

6.2 Effect of Penalty Intensity

The experiments of the previous section revealed a clear difference between the initialization strategies. Feasible and infeasible populations followed distinct search paths, but all results were obtained with the same penalty intensity ($\rho = 1$). This makes it difficult to separate the effect of the initialization method from the feasibility pressure imposed by the objective function (Equation 2).

Table 3: Final fitness results for Alternating initialization. All runs converged to the same fitness value for both EDA and EDA-TREE.

Instance	Best Fitness
NL8	58978
NL10	98073
NL12	187308
NL14	363230
NL16	527033

To evaluate the sensitivity of the penalty intensity, we considered three representative values of ρ , ranging from weak (0.1) to strong (0.8) penalization. We repeated the experiments using three penalty intensity values ($\rho = 0.1$, $\rho = 0.4$, and $\rho = 0.8$). The objective is to evaluate if reducing or increasing the contribution of constraint violations to the fitness function affects the ability of the algorithms to explore the search space and improve feasibility.

Changing the intensity of the penalty produced only minor variations in the final feasibility level. For most instances, the average number of constraint violations remained nearly unchanged across all tested values of ρ . The largest differences were observed for NL10 under EDA, where lower penalty values produced a smaller number of violations. Even in this case, the difference stayed below one violation on average. This effect can be observed in Figure 6, which illustrates the evolution of the mean number of violations for NL8 and NL10.

For the larger benchmark instances (NL12, NL14, and NL16), the influence of the penalty intensity was limited. Across all initialization strategies, EDA and EDA-Tree reached similar levels of infeasibility regardless of the value of ρ , as shown in Table 4.

Initialization influenced the search behavior more strongly than penalty intensity, but feasibility still being a problem for EDA and EDA-Tree. The next section investigates if the local search procedure can improve this behavior.

6.3 Role of Local Search

Since the previous experiments revealed persistent feasibility issues, local search was incorporated into the EDA and EDA-Tree configurations from Tables 2 and 3. To provide intensification with a lower computational cost, the local search procedure was applied every 50 generations with probability 0.1. These values were empirically chosen and kept fixed throughout all experiments and each experiment was repeated over 30 independent runs. The best hybrid configuration obtained for each benchmark instance was then compared against standalone local search, using the same local search procedure and computational budget. The results of this comparison are presented in Table 5.

All solutions reported in Table 5 are feasible, confirming the ability of the local search procedure to maintain and improve feasible solutions throughout the

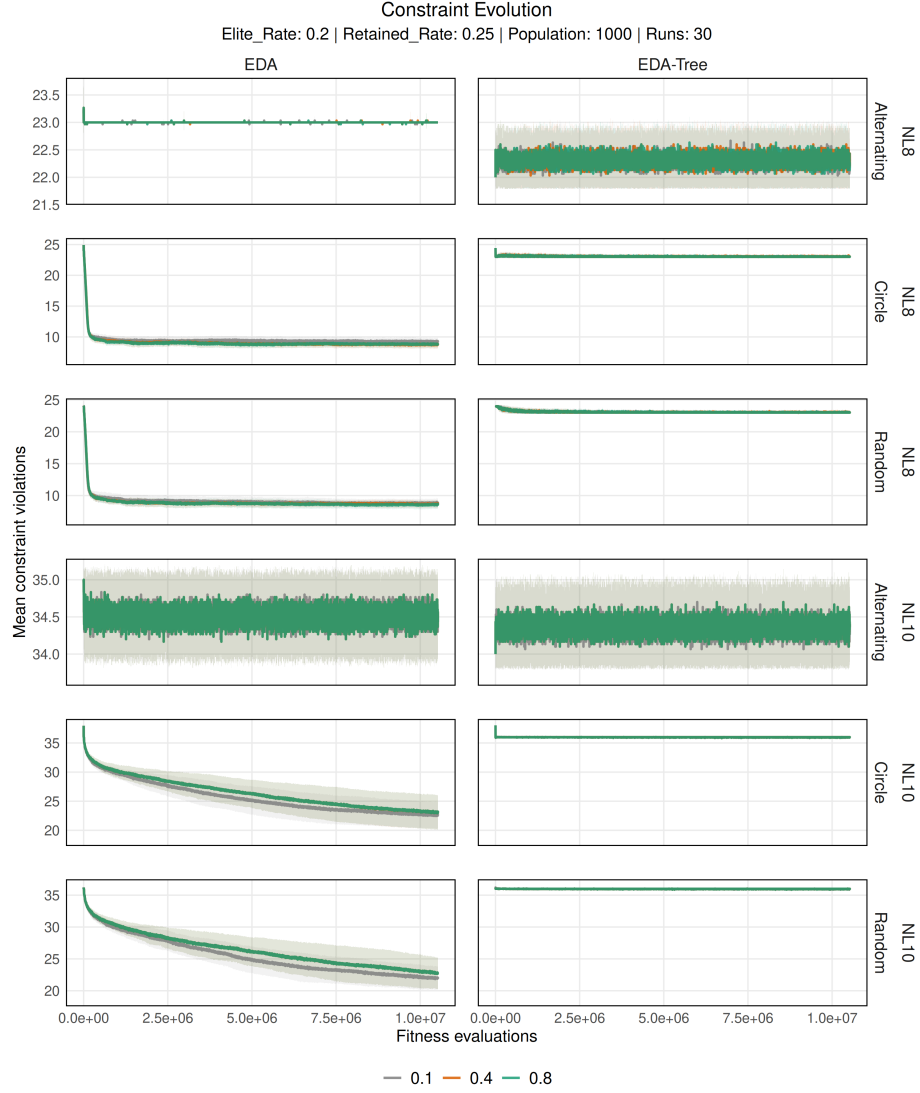


Fig. 6: Evolution of mean constraint violations for different penalty intensities. Significant differences were only observed in NL10 under EDA, where lower penalty intensity ($\rho = 0.1$) led to fewer constraint violations.

Table 4: Evolution of mean constraint violations for different penalty intensities.

Instance	Algorithm	Initialization	Penalty Intensity (ρ)		
			0.1	0.4	0.8
NL8	EDA	Alternating	23.00	23.00	23.00
		Circle	9.27	8.80	8.83
		Random	8.77	8.73	8.60
	EDA-Tree	Alternating	22.30	22.20	22.17
		Circle	23.00	23.10	23.00
		Random	23.00	23.00	23.00
NL10	EDA	Alternating	34.50	34.43	34.43
		Circle	22.57	23.13	23.13
		Random	21.93	22.73	22.73
	EDA-Tree	Alternating	34.37	34.40	34.40
		Circle	35.97	35.97	35.97
		Random	36.00	35.93	35.93
NL12	EDA	Alternating	48.00	48.00	48.00
		Circle	46.87	46.80	46.80
		Random	46.73	46.63	46.63
	EDA-Tree	Alternating	48.07	48.00	48.00
		Circle	49.90	49.93	49.93
		Random	50.00	49.97	49.97
NL14	EDA	Alternating	63.03	63.03	63.03
		Circle	63.40	63.63	63.63
		Random	63.70	63.50	63.50
	EDA-Tree	Alternating	63.63	63.67	63.67
		Circle	65.73	65.63	65.63
		Random	65.73	65.67	65.67
NL16	EDA	Alternating	80.27	80.20	80.20
		Circle	81.87	81.80	81.80
		Random	81.83	81.83	81.83
	EDA-Tree	Alternating	80.77	80.90	80.90
		Circle	83.37	83.37	83.37
		Random	83.40	83.47	83.47

Table 5: Comparison between the best EDA configuration and standalone local search.

Instance	Best EDA Configuration	EDA Mean	LS Mean	Difference (%)	p-value
NL8	Alternating + EDA + LS	48,644	50,895	+4.42	9.08e-12*
NL10	Random + EDA-Tree + LS	81,972	89,288	+8.19	<2.2e-16*
NL12	Alternating + EDA-Tree + LS	172,686	167,047	-3.37	0.6937
NL14	Alternating + EDA + LS	361,480	324,531	-11.38	3.69e-11*
NL16	Alternating + EDA + LS	527,033	475,424	-10.85	1.21e-12*

search. For the instances NL8 and NL10, the hybrid approaches outperformed standalone local search, obtaining improvements of 4.42% and 8.19%, respectively. The Wilcoxon rank-sum test indicated that these differences remained statistically significant after applying the Bonferroni correction ($\alpha = 0.01$, corrected $\alpha = 0.002$). In contrast, no statistically significant difference was observed for the medium-sized instance NL12.

For NL14 and NL16, standalone local search outperformed the best EDA based configurations, improving the average solution quality by more than 10%. Together with the results observed for NL8, NL10, and NL12, the influence of the probabilistic models became progressively smaller as the instance size increased. In larger instances, most of the improvement appears to come from the local search procedure instead of from the learned probabilistic models.

The comparison against standalone local search indicates that most of the performance gains obtained by the hybrid approaches originate from the local search procedure. While the probabilistic models contribute to the search in smaller instances, their impact became progressively smaller as the instance size increased. This pattern is reflected by the transition from significant improvements in NL8 and NL10 to comparable performance in NL12 and, then to the advantage of standalone local search in NL14 and NL16.

7 Conclusion

The Traveling Tournament Problem remains challenging for probabilistic model based optimization because feasibility and solution quality are strongly connected. In this work, we investigated how different initialization strategies affect the behavior of EDA and EDA-Tree, and if increasing the penalty associated with constraint violations can improve their ability to reach feasible schedules.

Among all factors considered, initialization had the strongest influence on the search process. Fully feasible initializations produced good schedules from the beginning, but often reduced exploration and consistently trapped both EDAs in the initial feasible basin. In contrast, infeasible and partially feasible initializations explored larger regions of the search space, but this additional exploration did not translate into feasible solutions.

Changing the penalty intensity produced a smaller effect. Stronger penalties increased the pressure for feasibility, but did not substantially alter the overall search behavior. The main challenge is not the objective function itself, but the ability of the probabilistic models to generate feasible solutions and preserve feasibility during the search.

The comparison with standalone local search revealed a different pattern, for the smaller benchmark instances, the hybrid approaches benefited from the combination of probabilistic modeling and local improvement. However, this advantage gradually disappeared as the instance size increased, for the largest instances, standalone local search produced better solutions. In these scenarios, the probabilistic models contributed little under the investigated settings.

In the end, these results suggest that the main challenge is not only learning probability distributions over solutions, but learning models capable of exploiting the structure of highly constrained scheduling problems. Future work will investigate alternative probabilistic representations and hybrid strategies that better balance exploration, feasibility, and local improvement.

References

1. Baluja, S., Davies, S.: Using optimal dependency-trees for combinatorial optimization: Learning the structure of the search space. Tech. rep. (1997)
2. Bengoetxea, E., Larrañaga, P., Bloch, I., Perchant, A.: Estimation of distribution algorithms: A new evolutionary computation approach for graph matching problems. In: International Workshop on Energy Minimization Methods in Computer Vision and Pattern Recognition. pp. 454–469. Springer (2001)
3. Bhattacharyya, R.: A note on complexity of traveling tournament problem. *Optimization Online* **2480** (2009)
4. Bhattacharyya, R.: Complexity of the unconstrained traveling tournament problem. *Operations Research Letters* **44**(5), 649–654 (2016)
5. Chow, C., Liu, C.: Approximating discrete probability distributions with dependence trees. *IEEE transactions on Information Theory* **14**(3), 462–467 (1968)
6. De Werra, D.: Scheduling in sports. *Studies on graphs and discrete programming* **11**, 381–395 (1981)
7. Di Gaspero, L., Schaerf, A.: A tabu search approach to the traveling tournament problem. In: Atti della Giornata di Lavoro del gruppo RCRA 2005: Analisi sperimentale e benchmark di algoritmi per l’Intelligenza Artificiale. pp. 23–27 (2005)
8. Du, Y., Li, J.q., Chen, X.l., Duan, P.y., Pan, Q.k.: Knowledge-based reinforcement learning and estimation of distribution algorithm for flexible job shop scheduling problem. *IEEE Transactions on Emerging Topics in Computational Intelligence* **7**(4), 1036–1050 (2022)
9. Easton, K., Nemhauser, G., Trick, M.: The traveling tournament problem description and benchmarks. In: Principles and Practice of Constraint Programming—CP 2001: 7th International Conference, CP 2001 Paphos, Cyprus, November 26–December 1, 2001 Proceedings 7. pp. 580–584. Springer (2001)
10. Gao, S., de Silva, C.: A modified estimation distribution algorithm based on extreme elitism. *Biosystems* **150**, 149–166 (12 2016). <https://doi.org/10.1016/j.biosystems.2016.10.001>

11. Goerigk, M., Hoshino, R., Kawarabayashi, K.i., Westphal, S.: Solving the traveling tournament problem by packing three-vertex paths. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. vol. 28 (2014)
12. Hassel, L., Verduin, K., Loyen, B., Thomson, S.L., Weise, T., van den Berg, D.: The traveling tournament problem: Constraint violations for different maxstreak values. *The Leading European Event on Bio-Inspired AI* p. 5 (2025)
13. Hauschild, M., Pelikan, M.: An introduction and survey of estimation of distribution algorithms. *Swarm and evolutionary computation* **1**(3), 111–128 (2011)
14. Khelifa, M., Boughaci, D.: A cooperative local search method for solving the traveling tournament problem. *Computing and Informatics* **37**(6), 1386–1410 (2018)
15. Larranaga, P., Bielza, C.: Estimation of distribution algorithms. In: *Handbook of Heuristics*, pp. 583–598. Springer (2025)
16. Loyen, B., Bart, D., Richoux, F., van den Berg, D.: The traveling tournament problem: Valid solutions are different across instance sizes. In: *International Joint Conference on Computational Intelligence*. pp. 491–503. Springer (2025)
17. Nakahata, G., Richoux, F., van den Berg, D., Aranha, C.: A constructive method to build many valid initial solutions for the traveling tournament problem. In: García-Sánchez, P., Díaz Álvarez, J., Murphy, A. (eds.) *Applications of Evolutionary Computation*. pp. 441–456. Springer Nature Switzerland, Cham (2026)
18. Ribeiro, C., Urrutia, S., de Werra, D.: *Combinatorial Models for Scheduling Sports Tournaments*. EURO Advanced Tutorials on Operational Research, Springer International Publishing (2023)
19. Richoux, F., Uriarte, A., Baffier, J.F.: ghost: A combinatorial optimization framework for real-time problems. *IEEE Transactions on Computational Intelligence and AI in Games* **8**(4), 377–388 (2016). <https://doi.org/10.1109/TCIAIG.2016.2573199>
20. Sun, Y., Yen, G.G., Yi, Z.: Reference line-based estimation of distribution algorithm for many-objective optimization. *Knowledge-Based Systems* **132**, 129–143 (2017)
21. Verduin, K., Thomson, S.L., van den Berg, D.: Too constrained for genetic algorithms. too hard for evolutionary computing. the traveling tournament problem. In: *Proceedings of the 15th International Joint Conference on Computational Intelligence ECTA - Volume 1*. pp. 246–257 (2023). <https://doi.org/10.5220/0012192100003595>
22. Xiang, C., Wu, Z., van Den Berg, D., Weise, T.: Randomized local search vs. nsga-ii vs. frequency fitness assignment on the traveling tournament problem. In: *IJCCI*. pp. 38–49 (2024)
23. Xu, Q., Liu, A., Yuan, X., Song, Y., Zhang, C., Li, Y.: Random mask-based estimation of the distribution algorithm for stacked auto-encoder one-step pre-training. *Computers & Industrial Engineering* **158**, 107400 (2021)
24. Zhao, J., Xiao, M.: A matching-based algorithm for the traveling tournament problem. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. vol. 39, pp. 26750–26758 (2025)
25. Zhou, B.H., Tan, F.: A self-adaptive estimation of distribution algorithm with differential evolution strategy for supermarket location problem. *Neural Computing and Applications* **32**(10), 5791–5804 (2020)